02 UNITY · 게임 프로토타입 개발 포트폴리오

귤 플랫포머

Orange Platformer — 3D 액션 플랫포머 1스테이지

> 1차 평가에서 만든 귤 캐릭터 · 귤 · 믹서기를 그대로 써서, 마리오 64 · 어 햇 인 타임처럼 다양한 액션으로 스테이지를 건너는 3D 플랫포머를 AI 코딩 도구로 개발했습니다.



훈련생	우정완 · 2026 대전 KNDA 게임 개발 과정
GitHub 저장소	https://github.com/wakhang/orange-platformer
실행 빌드 (PC)	Releases v1.0 · OrangePlatformer_Windows.zip (압축 해제 후 OrangePlatformer.exe)
엔진 · 기술	Unity 6 (6000.6.0f1) · URP 17.6 · Input System 1.20 · uGUI
AI 도구	OMP Studio 워크플로 (Claude Opus 5.5 · Sonnet 5.5 · GPT-6.1 Sol) + Unity · Blender · VARCO 3D MCP

01 기획 의도

콘셉트 · 핵심 재미 · 범위

> **귤 캐릭터가 과수원 섬을 건너며 귤 8개를 모으고, 뚜껑이 열린 믹서기 안으로 뛰어들면 클리어하는 3D 플랫포머**입니다. 1차 평가의 리소스 3종(캐릭터 · 귤 · 믹서기)을 그대로 써서 1차의 "게임 콘셉트"를 실제로 플레이할 수 있게 만드는 것이 목표입니다.

핵심 재미: 액션마다 지나가야 하는 구간

마리오 64 · 어 햇 인 타임처럼 **여러 가지 이동 액션을 익혀 쓰는 맛**을 핵심으로 잡았습니다. 각 구간은 해당 액션 없이는 통과할 수 없도록 수치로 막았습니다.

구간	장애	액션 없이 가능한 최대	필요한 액션
B 발판의 귤	높이 3 m	점프 1.6 m	이단 점프 (1.6 + 1.3 = 2.9 m)
C 수직갱 → D 고원	높이 6 m	이단 점프 2.9 m	벽 점프 3회 내외 (회당 약 1.6 m 상승)
E 높은 단	높이 3.5 m	2.9 m	웅크렸다 높이뛰기 (4.0 m)
F 넓은 틈	폭 8 m	달려서 점프 약 7 m	점프 + 다이브 (약 8.4 m)
G 과일 상자 2개	귤이 상자 안	점프·다이브로 파괴 불가	엉덩방아

게임 루프

시작 조작법 안내 Enter ▶ **이동·액션** 점프 · 다이브 벽 점프 · 엉덩방아 ▶ **수집** 귤 8개 (상자 속 2개) ▶ **목표 개방** 믹서기 뚜껑 열림 ▶ **승리** 믹서기 안으로 점프

- **실패:** 주스 바다에 빠지면 하트 1 감소 후 마지막 체크포인트에서 부활, 하트 3개를 모두 잃으면 게임오버.
- **범위:** 1스테이지에 집중, 3~5분 분량. 완성도(조작감 · 연출)를 우선했습니다.

1차 리소스 활용



귤 캐릭터 (플레이어, 1.0 m) — 1차 메시·리그 그대로



믹서기 (최종 골, 2.4 m) — 8개 모으면 뚜껑이 열림



귤 (수집 아이템, 0.5 m) + 새 VARCO 장식

02 개발 과정

이번 과제는 **OMP Studio의 워크플로**로 개발했습니다. 워크플로는 기획 · 코드 · UI · 3D 에셋 · 씬 구성 · 검증을 맡는 AI 에이전트들을 노드로 연결한 그래프입니다. 저는 요청문을 쓰고, 계획을 승인하고, 결과를 플레이해 보고 피드백을 주는 역할을 했습니다.

워크플로 구조 (game-dev-lean)



- 컴파일/테스트나 플레이 테스트가 실패하면 **수정 에이전트(Sonnet)**로 돌아가 고친 뒤 다시 검증합니다. 수정은 스크립트 카운터로 최대 4회까지만 돕니다.
- 코드 결함이 아니라 **검증 기준이 맞지 않는 경우**는 수정 에이전트가 "판단 필요"로 표시해 사람 승인 단계로 넘깁니다(1차 실행 후 추가한 개선).
- 게임플레이 · UI 에이전트는 코드만 쓰고, Unity 에디터 조작은 씬 구성 단계 하나가 맡도록 나눠서 에디터 충돌을 막았습니다.

진행 순서

#	단계	내용	방법 · 소요
1	워크플로 점검·개선	플러그인 비활성, 플레이 검증 단계 부재, 레벨 담당 부재 등 7가지 문제를 찾아 그래프를 고침	모의 실행 4가지로 검증
2	1차 실행: 게임 제작	기획서 작성 → 코드·UI 병렬 → 씬 구성 → 컴파일·테스트 → 플레이 테스트 3회 · 수정 4회	워크플로 · 54분
3	룩 개선	어두운 그림자 원인 분석, 따뜻한 조명 · 주스 바다 · 과수원 섬 배경	직접 요청 · Unity MCP
4	2차 실행: 다듬기	걷기 떨림 수정, VARCO 장식 8종 생성, 애니메이션 9종 추가	워크플로 · VARCO 32분 포함
5	애니메이션 축 보정	방향이 틀어진 클립 6개를 측정해 보정	Blender MCP
6	이펙트	이동·액션·수집·승리 등 이벤트별 파티클 이펙트	직접 요청 · 코드
7	빌드·제출	Windows 빌드, GitHub 저장소·릴리스, 문서	Unity · GitHub CLI

사용 도구

도구	역할
OMP Studio 워크플로	에이전트 그래프 실행 · 승인 단계 · 실행 기록(프롬프트·출력·오류) 보관
Unity MCP	AI가 에디터를 직접 조작: 임포트, 프리팹, 씬 배치, 플레이 모드 테스트, 스크린샷
Blender MCP · VARCO 3D MCP	장식 모델 생성(이미지 → 3D), 애니메이션 생성 · 리그 이전 · 축 보정

03 씬 구성

임포트 · 프리팹 · 레벨 · 조명 · 카메라 · 배경

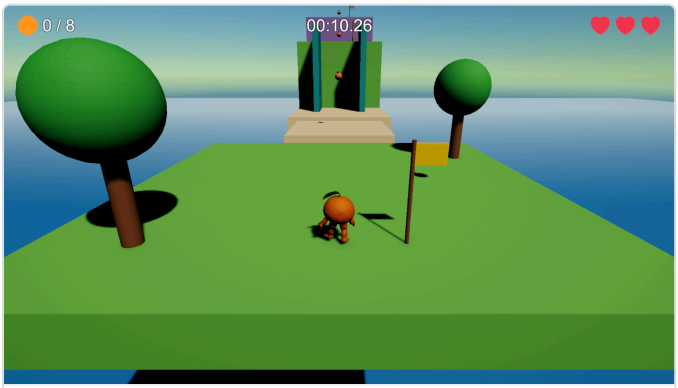


스테이지 전체: 시작 섬 → 이단 점프 발판 → 벽 점프 수직갱 → 고원 → 높은 단 → 다이브 틈 → 상자 → 믹서기 골. 주변은 과수원 섬과 오렌지주스 바다

에셋 임포트 · 프리팹화

에셋	출처	처리
Orange_Character · Orange_Prop · Mixer_Goal	1차 FBX 그대로	Generic 리그, Idle/Walk 루프, URP Lit 머티리얼에 텍스처 연결, 콜라이더 · 스크립트를 붙여 프리팹화
귤나무 · 과일 상자 · 주스팩 · 귤 반쪽 · 착즙기 · 표지판 · 꽃 덤불 · 빨대 깃발	VARCO 3D 새로 생성 (8종)	Blender로 크기·원점·면 수 정리 → 콜라이더 없는 Static 장식 프리팹 53개 배치. 부서지는 상자만 콜라이더 유지
발판 · 벽 · 체크포인트 · KillZone	Unity 기본 도형	Platform · WallJumpWall · Checkpoint · KillZone 등 프리팹 12종

조명 · 카메라 · 배경 개선



개선 전 그늘이 완전히 검고 배경이 단조로운 파란 바다



개선 후 따뜻한 3색 환경광, 부드러운 그림자, 과수원 섬과 주스 바다

- **조명:** 환경광 Trilight(하늘·지평선·바닥), 햇빛 1.5 · 그림자 진하기 0.55, 반사용 환경 데이터 생성.
- **후처리:** 톤매핑 Neutral, Bloom 0.35, 비네트 0.08, 채도 +12. **안개:** 복숭아색 선형 안개 90~300 m.
- **카메라:** 자체 제작 3인칭 궤도 카메라(거리 6 m, 피치 −20~65°, 벽 충돌 시 당겨짐, 리스폰 시 즉시 스냅).

04 게임플레이 구현

조작 · 액션 · 규칙

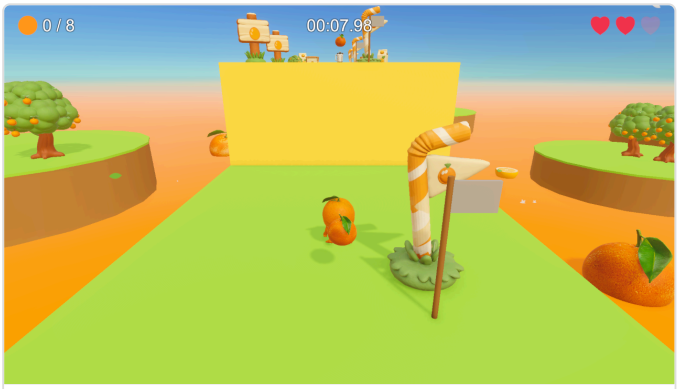
조작

동작	키보드 · 마우스	설명
이동 / 시점	WASD · 마우스	카메라 기준 이동, 마우스로 궤도 카메라 회전 (게임패드 스틱도 지원)
점프 · 이단 점프	Space (공중에서 한 번 더)	길게 누르면 높게(가변 점프), 코요테 타임 0.12초 · 점프 버퍼 0.15초로 손맛 보정
웅크리기 · 높이뛰기	Shift / C → Space	웅크린 채 점프하면 4 m 높이뛰기
다이브	E / 좌클릭 (공중)	앞으로 12 m/s 몸통 박치기 후 배로 미끄러짐
벽 슬라이드 · 벽 점프	청록 벽에 붙어 Space	미끄러지며 낙하 속도 3 m/s로 제한, 반대쪽으로 튕겨 오름
엉덩방아	공중에서 Shift / C	0.25초 정지 후 25 m/s로 내려찍어 상자를 부숨
일시정지 / 확인	Esc / Enter	일시정지 패널, 시작·재시작

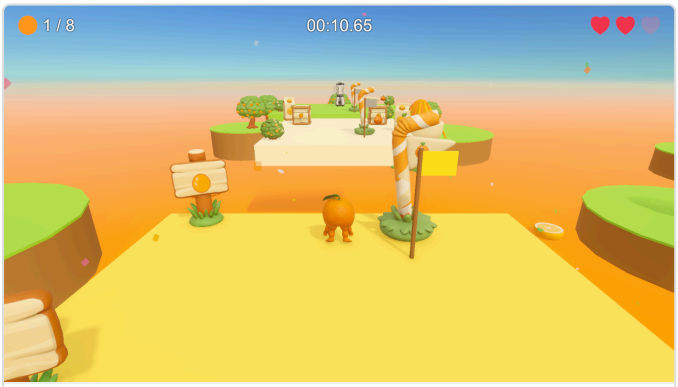
액션 구간



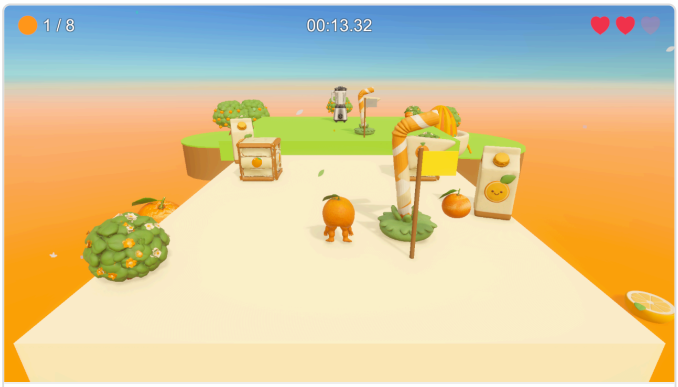
벽 점프 수직갱 (6 m): 청록 벽 사이를 번갈아 차고 오름



높은 단 (3.5 m): 웅크렸다 높이뛰기로만 오를 수 있음



다이브 틈 (8 m): 점프 + 다이브로 건넘



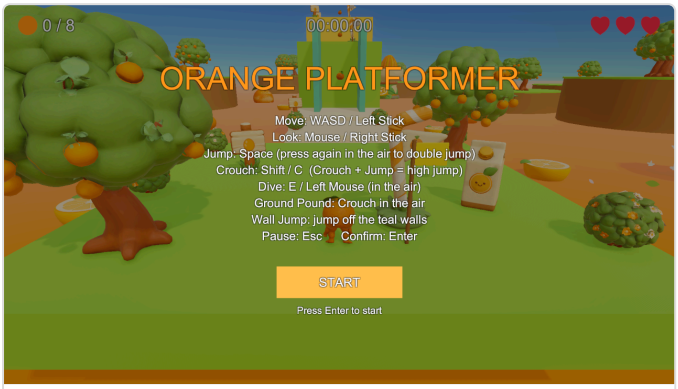
과일 상자: 엉덩방아로 부수면 안의 귤이 나옴

구현 방식

- **캐릭터 컨트롤러:** CharacterController 기반, 9개 상태(Grounded · Crouching · Airborne · WallSlide · Dive · DiveSlide · GroundPoundHang/Fall/Land)의 상태 기계. 물리는 FixedUpdate(50 Hz)에서 계산합니다.
- **수치 설계:** 점프 초속은 v = √(2gh)로 높이에서 역산(점프 1.6 m → 9.80 m/s). 모든 수치는 ScriptableObject `OrangePlatformerConfig`에서 조절합니다.
- **규칙:** 상태 전이 · 귤 · 하트 · 타이머는 Unity에 의존하지 않는 순수 C# 모델(`GameRulesModel`)로 분리해 단위 테스트했습니다.

05 UI 구현

UI는 게임 세션의 **이벤트(StateChanged · OrangesChanged · HeartsChanged · GoalOpened · RunEnded)를 구독**해서만 갱신됩니다. UI가 게임 로직을 직접 읽거나 바꾸지 않아서, 게임플레이와 UI를 서로 다른 에이전트가 동시에 만들 수 있었습니다.



시작 화면: 조작법 안내, START 버튼 / Enter



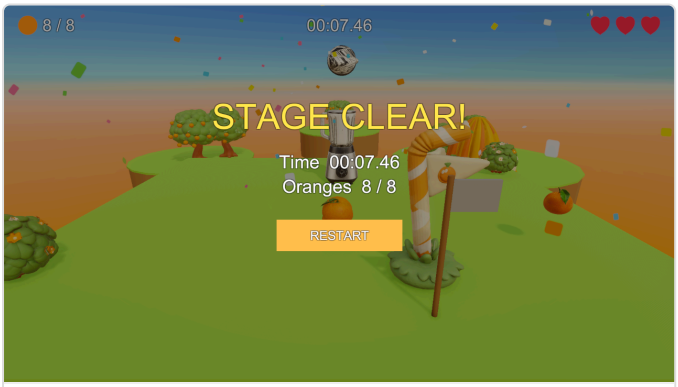
HUD: 귤 수(0/8) · 경과 시간 · 하트 3개



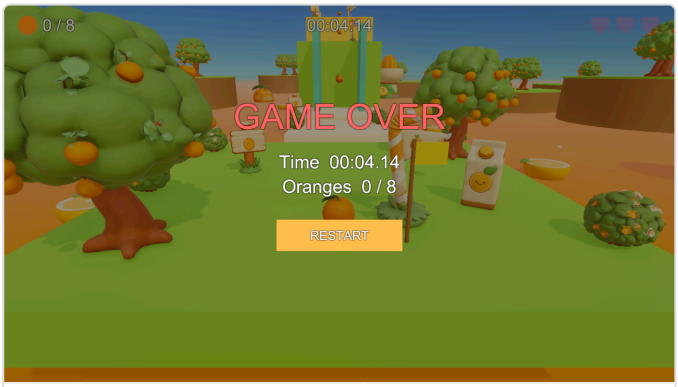
일시정지(Esc): 시간 정지 · RESUME / RESTART



8개 수집 → "The mixer is open! Jump in!" 안내 + 반짝이



클리어: 기록 시간 · 귤 수 · 재시작



게임오버: 하트 0 → 결과 화면 · 재시작

UI	구독 이벤트	동작
HudView	OrangesChanged · HeartsChanged · 매 프레임 시간	귤 수, 하트 아이콘, mm:ss.ff 타이머
IntroPanelView	StateChanged	Intro일 때만 표시, START → StartGame()
PausePanelView	StateChanged	Paused일 때 표시, RESUME · RESTART
ResultPanelView	RunEnded	승리/게임오버 결과(시간 · 귤 수), RESTART

06 연출: 애니메이션 · 이펙트

손맛 보강

캐릭터 애니메이션 9종

1차 모델의 메시 · 리그(본 27개)는 그대로 두고, VARCO 3D Animate로 만든 동작을 Blender에서 이 리그로 옮겼습니다. 상태 기계의 상태와 속도로 클립을 고릅니다.

Idle	Walk	Run	Jump	Fall	Dive	WallSlide	GroundPound	Victory
정지	~4.5 m/s	4.5 m/s~	상승	하강	다이브	벽 미끄럼	엉덩방아	승리

- VARCO에 없던 WallSlide는 Blender에서 직접 포즈를 잡았고, GroundPound는 가장 비슷한 웅크린 점프 동작으로 대신했습니다.
- 점프 시 늘어남 · 착지 시 찌그러짐, 이단 점프 앞구르기, 높이뛰기 720° 회전 같은 절차적 연출을 애니메이션 위에 더했습니다.

파티클 이펙트



엉덩방아 충격파 · 수집 주스 물방울 · 체크포인트 컨페티 (슬로 모션 캡처)



먼지 퍼프 · 나무 파편 · 반짝이 · 컨페티

상황	이펙트
걷기 · 달리기 / 점프 · 착지	발밑 먼지(속도 비례) / 원형 먼지 퍼프(낙하 속도 비례)
이단 점프 · 높이뛰기	발밑 링 + 반짝이 + 귤 껍질 조각 / 위로 솟는 먼지 줄기
다이브 · 벽 슬라이드	속도선 트레일, 미끄럼 먼지 / 벽 접촉점 먼지
엉덩방아	충격파 링 · 파편 · 약한 카메라 흔들림
수집 · 상자 · 체크포인트	주스 물방울 / 나무 파편 / 컨페티
목표 개방 · 승리 · 낙사 · 부활	믹서기 반짝이 기둥 / 컨페티 폭발 / 주스 물보라 / 등장 퍼프
배경	떠다니는 꽃잎 · 잎, 주스 바다 반짝임

모양별 파티클 시스템 9개를 미리 만들고 이벤트 때 입자만 내보내는 구조라, 이펙트마다 오브젝트를 생성하지 않습니다(GC 할당 없음). 텍스처는 코드로 그린 3×2 아틀라스 한 장입니다.

07 AI 활용 개발 ①

아래는 제가 실제로 보낸 요청문과, AI가 수행한 작업 · 결과입니다. 워크플로 실행 기록에는 각 에이전트의 프롬프트와 출력이 모두 남아 있습니다.

#	요청문	AI가 수행한 작업 · 수정 내용
1	“이번에는 이 OMP스튜디오의 워크플로우를 이용해서 만들건데 지금 한번 점검해볼래”	워크플로 검증 · 모의 실행. 필요한 플러그인이 꺼져 실행 불가, 1차 FBX를 가져오는 경로 없음 등을 보고
2	“워크플로우 자체만 봤을때 과정이 괜찮아? / 원본 고쳐줘 다 개선해줘”	"실제 플레이 검증이 없다"는 구조 문제를 찾아 플레이 테스트 노드, 씬 구성 단계, 수정 횟수 카운터, 사람 플레이 확인 단계를 추가
3	“게임은 3D플렛포머로 마리오나 어햇인타임같은 다양한 액션조작이 있었으면 좋겠네”	Unity 프로젝트 생성 후 워크플로 실행: 기획서(액션별 차단 구간 수치 검증 포함), 캐릭터 컨트롤러 · 세션 · 레벨 코드, UI, 씬, EditMode 테스트, 플레이 테스트 19항목
4	“화면이 어두운부분이 너무 많고 그림자가 너무 어둡고 캐릭터와 귤에 어울리게 배경을 좀 바꿔줘”	원인(조명 설정 에셋이 없어 환경광이 계산되지 않음)을 찾아 조명 재구성, 주스 바다 · 과수원 섬 배경 제작
5	“걸어다닐때 좀 덜덜 떨리는 거같고, 바르코 3d로 모델 더 생성해도되니까 더 꾸며줬으면 좋겠어 애니메이션도”	떨림 원인 진단 → 렌더 보간 구현 · 수치 검증, VARCO 장식 8종 생성 · 배치, 애니메이션 9종
6	“축이 안맞는 애니매이션들이 좀 있어 / 축 문제만 해결하자”	클립별 몸 방향을 측정해 Run(+47°) 등 6개 클립의 루트 회전 보정
7	“1차평가때 쓴 모델은 그대로 써야하는데, 지금 다른 플레이어모델 쓰지않았음?”	메시 정점 해시 · 본 위치 · 텍스처를 1차 파일과 비교해 동일 모델임을 확인
8	“이펙트가 없어서 좀 슴슴하네 이펙트 만들어서 넣어줘”	이벤트 기반 파티클 이펙트 시스템(OrangeVfx) 구현, 이펙트 20종

워크플로 실행 기록

실행	노드별 결과	결과
1차 (게임 제작)	계획 5.7분 → 코드 5.2분 ‖ UI 1.3분 → 씬 구성 11.5분 → 컴파일·테스트 5회 → 플레이 테스트 3회 → 수정 4회	18/19 통과 후 수정 한도 도달
2차 (다듬기)	계획 6.1분 → 코드 1.3분 ‖ 3D 에셋 32.1분(VARCO 생성 31회 · Animate 20회) → 씬 구성 6.2분 → 플레이 테스트 2회	21/22 통과, 남은 1건은 "판단 필요"로 승인

08 AI 활용 개발 ②

#	증상	원인	해결 · 검증
1	테스트 어셈블리 컴파일 오류	테스트 asmdef가 TestRunner를 중복 참조	중복 참조 제거 → 컴파일 오류 0
2	낭떠러지에서 걸어 내려간 뒤에도 이단 점프 가능	떨어질 때 이단 점프 기회를 소모하지 않음	`OnLeftGround`에서 소모, 코요테 점프 시 재충전 → 플레이 테스트 6번 통과
3	상자를 엉덩방아로 부수면 그 자리에서 멈춤	부순 같은 프레임에 CharacterController가 착지로 판정	부순 프레임은 착지 처리 건너뛰기 (`poundBrokeThisMove`) → 12번 통과
4	믹서기에 들어갈 수 없어 승리 불가	용기의 메시 콜라이더 윗면이 입구를 막음	속 빈 박스 콜라이더(벽 4개 + 바닥)로 교체 → 실제 점프로 승리 확인
5	수정 루프가 한도까지 돌고 실패로 종료	코드 결함이 아니라 "낙사 1초 뒤 부활" 기준이 낙하 시간을 고려하지 않음	낙사 판정 높이를 −9 → −2.5 m로 올려 부활을 빠르게. 워크플로에 "판단 필요" 경로 추가
6	그늘이 완전히 검음	씬에 조명 설정 에셋이 없어 환경광이 0	조명 설정 생성 · 3색 환경광 · 환경 데이터 생성 → 그림자 진하기 0.55
7	걸을 때 캐릭터가 덜덜 떨림	이동은 초당 50번(FixedUpdate)인데 화면은 제한 없이 그려지고 카메라가 매 프레임 따라감	물리는 그대로 두고 화면 위치만 보간. 84프레임 모두 6±0.6 m/s, 화면 흔들림 2 px 이하(보간을 끄면 0% 만족)
8	상자 외형 교체 후 부서져도 상자가 보임	스크립트가 자기 렌더러만 숨기는데 새 메시를 자식으로 두려 함	메시를 상자 자신의 MeshFilter에 넣음 → 파괴 후 렌더러 0개 확인
9	달리기 · 점프 등에서 몸이 옆을 봄	다른 캐릭터에서 만든 동작을 옮기며 루트 방향이 틀어짐(Run +47°, Jump 25° 비틀림, Fall 40° 뒤로 젖힘)	클립별 골반 방향 · 기울기를 측정해 루트 회전만 보정, 반복 이음새 오차 0
10	이펙트가 모두 네모 조각으로 나옴	텍스처 시트의 시작 프레임 값이 프레임 번호가 아닌 0~1 비율	(번호 + 0.5) / 6으로 변경 → 모양별 표시 확인

배운 점

- AI 코드는 **컴파일 · 단위 테스트가 통과해도 실제 플레이에서 깨질 수 있었습니다**(2 · 3 · 4번). 그래서 플레이 모드에서 입력을 넣어 확인하는 검증 단계를 워크플로에 넣었습니다.
- "떨린다", "어둡다" 같은 감각적인 문제도 **수치로 측정**(프레임별 속도, 화면 좌표, 클립별 각도)한 뒤 고치니 원인을 정확히 찾을 수 있었습니다.

09 주요 코드 설명 ①

> 실제 소스를 **수정 없이 그대로** 옮겼습니다. 왼쪽 숫자는 줄 번호, 초록 글씨는 소스의 원래 주석, : 는 생략입니다.

규칙 모델 — GameRulesModel

Assets/Scripts/OrangePlatformer/Gameplay/Session/GameRulesModel.cs · 줄 47–71 (원문 그대로)

```
47 /// <summary>Playing 중에만 집계. 필요 개수에서 상한 처리하고, 처음 도달한 순간에만 true 반환(목표 개방).</summary>
48 public bool CollectOrange()
49 {
50     if (State != GameState.Playing) return false;
51     if (Oranges < Required) Oranges++;
52     if (!GoalOpen && Oranges >= Required)
53     {
54         GoalOpen = true;
55         return true;
56     }
57     return false;
58 }
59
60 /// <summary>Playing 중에만 처리. 하트 1 감소, 0이 되면 GameOver 상태로 전이.</summary>
61 public Outcome PlayerFell()
62 {
63     if (State != GameState.Playing) return Outcome.Ignored;
64     if (Hearts > 0) Hearts--;
65     if (Hearts <= 0)
66     {
67         State = GameState.GameOver;
68         return Outcome.GameOver;
69     }
70     return Outcome.Respawn;
71 }
```

- Unity API를 쓰지 않는 **순수 C# 클래스**라 씬 없이 EditMode 테스트로 규칙을 검증합니다.
- `CollectOrange`는 8개를 **처음** 채운 순간에만 `true`를 돌려줘, 뚜껑 열림 이벤트가 한 번만 나갑니다.
- `PlayerFell`은 결과를 `Outcome`(무시 · 부활 · 게임오버)으로 돌려주고, 실제 부활은 세션이 처리합니다. 두 메서드 모두 먼저 `State`를 검사해 일시정지 · 결과 화면 중에는 규칙이 바뀌지 않습니다.

세션 — 규칙 결과를 이벤트로 알리기

Assets/Scripts/OrangePlatformer/Gameplay/Session/GameSession.cs · 줄 132–163 (원문 그대로)

```
132 public void PlayerFell()
133 {
134     // 리스폰 대기 중에는 KillZone이 반복 호출해도 무시
135     if (respawnPending) return;
136
137     GameRulesModel.Outcome outcome = model.PlayerFell();
138     if (outcome == GameRulesModel.Outcome.Ignored) return;
139
140     HeartsChanged?.Invoke(model.Hearts, model.MaxHearts);
141
142     if (outcome == GameRulesModel.Outcome.GameOver)
143     {
144         // 모델이 이미 GameOver로 전이했으므로 상태 적용 후 결과 발행
145         OnStateChanged(GameState.Playing);
146     }
147     else
148     {
149         respawnPending = true;
150         if (player != null) player.ControlsEnabled = false;
151         StartCoroutine(RespawnRoutine());
152     }
153 }
154
155 private IEnumerator RespawnRoutine()
156 {
157     yield return respawnWait;
158     respawnPending = false;
159     if (player != null) player.Respawn(checkpointPosition, checkpointRotation);
160     // 대기 중 일시정지/종료 상태가 되었다면 제어는 상태 적용 쪽에서 관리
161     if (model.State == GameState.Playing && player != null) player.ControlsEnabled = true;
162     PlayerRespawned?.Invoke();
163 }
```

- 규칙 모델의 결과를 **이벤트**로 알립니다(여기서는 `HeartsChanged`, 수집 시에는 `OrangesChanged` · `GoalOpened`). UI · 이펙트 · 믹서기는 이 이벤트만 구독합니다.
- `respawnPending`으로 KillZone이 매 프레임 호출해도 하트는 한 번만 줄고, 0.6초 뒤 코루틴이 체크포인트로 옮깁니다(대기 중 일시정지되면 조작을 켜지 않음, 161줄).

09 주요 코드 설명 ②

점프 높이 → 초기 속도

Assets/Scripts/OrangePlatformer/Gameplay/Player/PlayerMovementMath.cs · 줄 8–12 (원문 그대로)

```
/// <summary>점프 높이 h를 만드는 초기 수직 속도: v = sqrt(2*g*h).</summary>
public static float JumpSpeed(float gravity, float height)
{
    return Mathf.Sqrt(2f * Mathf.Max(0f, gravity) * Mathf.Max(0f, height));
}
```

- 기획서의 점프 **높이**(1.6 m · 이단 1.3 m · 높이뛰기 4.0 m)를 설정값으로 쓰고 초기 속도는 v = √(2gh)로 계산합니다(1.6 m, g = 30 → 9.80 m/s). 그래서 "6 m 벽은 이단 점프 2.9 m로 못 넘는다"처럼 구간 설계를 숫자로 확인했습니다.

공중 상태 — 액션 우선순위와 가변 점프

Assets/Scripts/OrangePlatformer/Gameplay/Player/PlayerController.cs · 줄 304–348 (원문 그대로)

```
    if (controlsEnabled)
    {
        // 점프 우선순위: 코요테 지상 점프 > 벽 점프 > 더블 점프
        if (timers.HasBufferedJump && timers.CanCoyoteJump)
        {
            timers.ConsumeBuffered();
            StartJump(JumpKind.Single);
            return;
        }
        if (timers.HasBufferedJump && wallContact)
        {
            timers.ConsumeBuffered();
            StartJump(JumpKind.Wall);
            return;
        }
        if (jumpPressed && doubleJumpAvailable && !highJumpAirtime)
        {
            timers.ConsumeBuffered();
            StartJump(JumpKind.Double);
            return;
        }

        if (!sliding)
        {
            // 다이브: 공중 1회, 조향/점프/그라운드파운드 불가
            if (divePressed && !diveUsed)
            {
                StartDive();
                return;
            }
            // 그라운드 파운드
            if (crouchPressed)
            {
                StartGroundPound();
                return;
            }
        }
    }

    // 가변 점프: 상승 중 점프 버튼을 떼면 상승 속도를 한 번만 감쇠 (단일/더블 점프만)
    if (jumpCutAllowed && velocity.y > 0f && !jumpHeld)
    {
        velocity.y *= config.jumpCutMultiplier;
        jumpCutAllowed = false;
    }
```

- 점프 키를 누르면 **코요테 점프 → 벽 점프 → 이단 점프** 순서로 판단합니다. 발판 끝에서 막 떨어진 직후 누르면 이단 점프를 아끼고 일반 점프가 나갑니다.
- 다이브 · 엉덩방아는 벽에 붙어 있지 않을 때만 되고, 다이브는 공중 1회(`diveUsed`)입니다.
- **가변 점프**(343–348줄): 올라가는 중 버튼을 떼면 상승 속도를 한 번 줄여, 짧게 누르면 낮게 · 길게 누르면 높게 뜁니다.
- 이어지는 350–370줄은 공중 조향(벽 점프 직후 0.2초 입력 잠금), 높이뛰기 중 수평 속도 제한, 벽 슬라이드 전이입니다.

09 주요 코드 설명 ③

렌더 보간 — 걷기 떨림 제거

Assets/Scripts/OrangePlatformerPolish/Gameplay/PlayerRenderInterpolator.cs · 줄 46–75 (원문 그대로)

```
46 // 순서 -200: PlayerController.FixedUpdate보다 먼저 실행되어 틱 직전 위치를 저장한다.
47 private void FixedUpdate()
48 {
49     prevTick = transform.position;
50 }
51
52 private void LateUpdate()
53 {
54     Vector3 curr = transform.position;
55     Vector3 render;
56     if (config == null || !config.interpolationEnabled)
57     {
58         render = curr;
59     }
60     else if (snap || RenderInterpolation.IsTeleport(prevTick, curr, config.teleportSnapDistance))
61     {
62         prevTick = curr;
63         render = curr;
64     }
65     else
66     {
67         float a = RenderInterpolation.Alpha(Time.time, Time.fixedTime, Time.fixedDeltaTime);
68         render = Vector3.Lerp(prevTick, curr, a);
69     }
70     snap = false;
71     RenderPosition = render;
72
73     if (visual != null)
74         visual.localPosition = baseLocalPos + transform.InverseTransformVector(render - curr);
75 }
```

- 이동 계산은 FixedUpdate(초당 50번)인데 화면은 더 자주 그려져서 캐릭터가 덜덜 떨려 보였습니다.
- 실행 순서 −200으로 물리 계산 **직전 위치**를 저장하고(49줄), 화면을 그릴 때 직전·현재 위치 사이를 시간 비율 a만큼 보간합니다(67–68줄). 보이는 모델(visual)만 옮기므로 충돌 · 점프 수치는 그대로입니다.
- 리스폰 같은 순간이동은 보간하면 미끄러져 보여서 바로 맞춥니다(60–64줄). 검증: 걷기 84프레임 모두 6±0.6 m/s, 화면 흔들림 2 px 이하.

HUD — 이벤트 구독으로만 갱신

Assets/Scripts/OrangePlatformer/UI/HudView.cs · 줄 62–73, 110–124 (원문 그대로)

```
62 protected override void OnAttached(IGameSession s)
63 {
64     s.OrangesChanged += OnOranges;
65     s.HeartsChanged += OnHearts;
66     s.GoalOpened += OnGoalOpened;
67     s.RunEnded += OnRunEnded;
68     s.StateChanged += OnState;
69     OnOranges(s.OrangesCollected, s.OrangesRequired);
70     OnHearts(s.Hearts, s.MaxHearts);
71     _lastCs = -1;
72     _timerText.text = UITheme.FormatTime(s.ElapsedSeconds);
73 }
      ⋮
110 void Update()
111 {
112     if (_toast.enabled && Time.unscaledTime >= _toastEnd) _toast.enabled = false;
113
114     // 타이머는 이벤트가 없으므로 Playing 중에만, 표시값(1/100초)이 바뀔 때만 갱신
115     if (Session != null && Session.State == GameState.Playing)
116     {
117         int cs = (int)(Session.ElapsedSeconds * 100f);
118         if (cs != _lastCs)
119         {
120             _lastCs = cs;
121             _timerText.text = UITheme.FormatTime(Session.ElapsedSeconds);
122         }
123     }
124 }
```

- HUD는 세션 이벤트를 구독해 귤 수 · 하트 · 목표 안내를 바꿉니다. 게임 로직을 직접 읽지 않아 UI와 게임플레이를 서로 다른 에이전트가 동시에 만들 수 있었습니다.
- 이벤트가 없는 타이머만 Update에서 그리되, **표시값(1/100초)이 바뀔 때만** 문자열을 새로 만들어 매 프레임 메모리 할당을 피했습니다.

09 주요 코드 설명 ④

이펙트 — 오브젝트 생성 없이 방출

Assets/Scripts/OrangePlatformer/Gameplay/Vfx/OrangeVfx.cs · 줄 208–216, 413–422 (원문 그대로)

```
208 private void OnGroundPoundImpact()
209 {
210     Emit(ringPs, Feet, Vector3.zero, 3.6f, 0.45f, Dust);
211     Emit(ringPs, Feet + Vector3.up * 0.02f, Vector3.zero, 2.2f, 0.3f, Sparkle);
212     PuffRing(Feet, 18, 4f, 0.5f);
213     Burst(chunkPs, Feet + Vector3.up * 0.2f, 10, 3.5f, 0.12f, 0.7f, Wood, 3f);
214     Burst(starPs, Feet + Vector3.up * 0.3f, 8, 3f, 0.25f, 0.4f, Sparkle, 1.5f);
215     Shake(0.22f, 1f);
216 }
        ⋮
413 private void Emit(ParticleSystem ps, Vector3 pos, Vector3 vel, float size, float life, Color32 color)
414 {
415     ep.position = pos;
416     ep.velocity = vel;
417     ep.startSize = size * sizeScale;
418     ep.startLifetime = life * lifeScale;
419     ep.startColor = color;
420     ep.rotation = Random.Range(0f, 360f);
421     ps.Emit(ep, 1);
422 }
```

- 엉덩방아 착지 이벤트가 오면 충격파 링 · 먼지 · 파편 · 반짝이를 내보내고 카메라를 0.22초 흔듭니다.
- `Emit`은 미리 만든 파티클 시스템 9개에 입자만 추가합니다. 이펙트마다 GameObject를 만들지 않아 GC 할당이 없고, 크기 · 수명 배율은 인스펙터에서 조절합니다.

단위 테스트 — 규칙 검증

Assets/Scripts/OrangePlatformer/Tests/GameRulesModelTests.cs · 줄 26–55 (원문 그대로)

```
26 [Test]
27 public void Timer_CountsOnlyWhilePlaying()
28 {
29     var m = new GameRulesModel(8, 3);
30     m.Tick(1f);
31     Assert.AreEqual(0f, m.Elapsed);
32     m.StartGame();
33     m.Tick(1.5f);
34     Assert.AreEqual(1.5f, m.Elapsed, 1e-5f);
35     m.Pause();
36     m.Tick(5f);
37     Assert.AreEqual(1.5f, m.Elapsed, 1e-5f);
38     m.Resume();
39     m.Tick(0.5f);
40     Assert.AreEqual(2f, m.Elapsed, 1e-5f);
41 }
42
43 [Test]
44 public void Hearts_TwoRespawnsThenGameOver()
45 {
46     var m = Playing(8, 3);
47     Assert.AreEqual(GameRulesModel.Outcome.Respawn, m.PlayerFell());
48     Assert.AreEqual(2, m.Hearts);
49     Assert.AreEqual(GameRulesModel.Outcome.Respawn, m.PlayerFell());
50     Assert.AreEqual(1, m.Hearts);
51     Assert.AreEqual(GameState.Playing, m.State);
52     Assert.AreEqual(GameRulesModel.Outcome.GameOver, m.PlayerFell());
53     Assert.AreEqual(0, m.Hearts);
54     Assert.AreEqual(GameState.GameOver, m.State);
55 }
```

- 규칙 모델을 씬 없이 바로 만들어 시간 흐름 · 하트 감소 · 게임오버 전이를 확인합니다. 타이머는 일시정지 중에 멈춰야 한다는 규칙도 숫자로 검사합니다.
- AI가 코드를 고칠 때마다 워크플로의 컴파일+테스트 단계가 이 테스트들을 자동으로 돌렸습니다(현재 28개 모두 통과).

10 결과물 · 검증

제출물

항목	위치
GitHub 저장소	github.com/wakhang/orange-platformer — Unity 프로젝트 전체, 기획서(Docs/Design), 소스 아트(ArtSource)
실행 빌드 (Windows 64bit)	Releases v1.0 · OrangePlatformer_Windows.zip → OrangePlatformer.exe
기획서	Docs/Design/OrangePlatformer.md (규칙 · 수치 · 구간 차단 검증), OrangePlatformerPolish.md (떨림 · 애니메이션)

검증

검증	결과
컴파일 + EditMode 테스트	**28 / 28 통과 (규칙 모델 · 이동 수학 · 애니메이션 상태 · 보간)**
플레이 모드 플레이 테스트 (AI QA)	22개 수용 기준 중 21개 통과. 남은 1개는 기준 쪽 문제(가변 점프로 탭 점프가 0.6초 전에 착지)로 판단해 승인
떨림 수치 검증	걷기 84프레임 모두 6±0.6 m/s, 화면 흔들림 2 px 이하
빌드 실행	Windows 빌드 실행 15초 동안 정상 동작, 로그 오류 0

평가 기준 대응

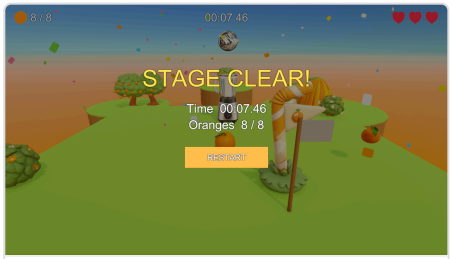
평가 영역	수행준거	근거
씬 구성	에셋 임포트 · 프리팹화 · 씬 구성	03장: 1차 FBX 3종 + VARCO 8종, 프리팹 12종 · 장식 53개
씬 구성	조명 · 카메라 · 배경	03장: 조명 개선 전·후, 궤도 카메라, 과수원 섬 · 주스 바다
게임플레이	플레이어 조작(이동 · 카메라)	04장: 7가지 액션 · 궤도 카메라, 08장 7번 떨림 해결
게임플레이	목표 · 획득 · 승리/실패	01 · 04장: 귤 8개 → 뚜껑 → 승리, 낙사 → 하트 → 게임오버
UI	게임 상태와 연동	05장: 이벤트 구독 기반 HUD · 시작 · 일시정지 · 결과 화면
AI 활용	프롬프트 · 수정 내용 기록	02 · 07장: 워크플로 구조, 요청문 8건, 실행 기록
AI 활용	오류 분석 · 해결, 주석 설명	08장: 오류 10건, 09장 ①~④: 주요 코드 원문 8곳 + 줄 번호 · 설명
결과물	실행 빌드 · 플레이 영상	10장: 빌드 v1.0



벽 점프 구간



목표 개방



스테이지 클리어